Telematics Platform User Guide

Update Package Generation

Document Information	
Filename	TP_F101U-00040_User_Guide_Update_Package_Generation
Status	Release
Revision Date	Apr 8, 2021
Version	05.00
Document ID	TP_F101U-00040

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Table of contents

1 Introduction 4
1.1 Notational Conventions 4
2 Setup 5
2.1 Tools/Equipment Needed 5
2.2 Setup Instructions 5
2.3 Setup Troubleshooting 5
3 Features/Components 5
3.1 Software Loading Update Package Generation 5
3.1.1 Overview 5
3.1.1.1 Platform Content Overview 5
3.1.1.2 Product Expectations 5
3.1.1.3 Extending Platform Functionality 6
3.1.2 Setup 6
3.1.2.1 Tools/Equipment Needed 6
3.1.2.2 Setup Instructions 6
3.1.2.3 Setup Troubleshooting 6
3.1.3 Usage 6
3.1.3.1 Basic Instructions 6
3.1.3.2 Detailed Instructions 7
3.1.3.2.1 Script set_env.sh 8
3.1.3.2.2 Script generate_diffs.sh 11
3.1.3.2.3 Script create_xml_descriptor.sh 13
3.1.3.2.4 Script gen_update_descriptor.sh 14
3.1.3.2.5 Script sign_files.sh 15
3.1.3.2.6 Script create_update_package.sh 17
3.1.3.2.7 Putting it all together 18
3.1.3.3 Preparing package for persistence update agent 19
3.1.3.3.1 Using “*.tar.gz” archives for persistence update 19
3.1.3.3.2 Using XML files for persistence update 19
3.1.3.4 Testing/Error Reporting 20
3.2 Software Download Blocks Generation 20
3.2.1 Overview 20
3.2.1.1 Platform Content Overview 20
3.2.1.2 Product Expectations 20
3.2.1.3 Extending Platform Functionality 20
3.2.2 Setup 20
3.2.2.1 Tools/Equipment Needed 20
3.2.2.2 Setup Instructions 21

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.2.3 Setup Troubleshooting 21
3.2.3 Usage 21
3.2.3.1 Basic Instructions 21
3.2.3.2 Detailed instructions 21
3.2.3.2.1 Parameters 21
3.2.3.2.2 Blocks config file 21
3.2.3.2.3 Executing the script 22
3.2.3.3 Testing/Error Reporting 23
4 Known Issues 23
5 FAQs 23
6 Troubleshooting/Tips and Tricks 24
7 References 24
7.1 Abbreviations/Acronyms/Definitions 24
7.2 Platform Documentation 24
7.3 Website links 24
8 Revision History 25

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides usage information for Update Package Generation. The feature is part of two other features:

- Telematics Platform Software Loading framework that includes processes running on the applications processor within the System on Chip of the Network Access Device of a Telematics Control Unit.
- The Software Development Kit for the Telematics Platform.

This guide focuses on providing an overview on how the Project should use the tools provided in the Software Development Kit to generate an update package file to be used for the Software Loading framework.

The definition of an update package is explained with detail in TP_F101U-00028.

The definition of content type is explained with detail in TP_F100W-00015.

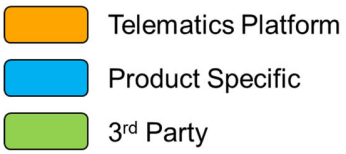
The installation of the Software Development Kit is explained with detail in TP_F101U-00035.

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



© Continental Automotive GmbH 2019
Confidential and Proprietary

2 Setup

2.1 Tools/Equipment Needed

The required tools should be included as part of the Software Development Kit for the Telematics Platform. Follow the instructions provided by TP_F101U-00035 in order to set the appropriate environment. The tools provided for the Update Package Generation process can run only in machines with Linux-based Operation Systems. In particularly, it has only been executed in Ubuntu 16.04.6 LTS but it should not be limited to that Operating System.

2.2 Setup Instructions

The details about the setup of the different hardware that can use the Software loading Manager or required for the development of a particular Update Agent or Download Client is beyond the scope of this guide. The details about the installation and usage of the Software Development Kit and the compilation of binaries is beyond the scope of this guide as well. This document just covers instructions to successfully generate an Update Package in a Linux-based Operating System. For that, the following component(s) should be installed in the host computer:

- /bin/bash executable

2.3 Setup Troubleshooting

Not applicable

3 Features/Components

3.1 Software Loading Update Package Generation

3.1.1 Overview

3.1.1.1 Platform Content Overview

The Software Loading feature provides below set of bash scripts for the update package generation process.

- **set_env.sh**: validates and creates the required set of environment variables to be used.
- **generate_diffs.sh**: Generates a diff binary file. This script is optional since it is only required for generating diff update images.
- **create_xml_descriptor.sh**: Creates individual xml files with update package metadata. Users can manually modify the generated xml.
- **gen_update_descriptor.sh**: Generates a consolidated update package metadata file based on individual ones.
- **sign_files.sh**: Generates signature files for file integrity checks.
- **create_update_package.sh**: Generates the update package in ISO format.

The Software Loading Update Package Generation Scripts are included as part of the Telematics Platform Software Development Kit.

3.1.1.2 Product Expectations

It is expected that users of Telematics Platform Software Loading Update Package Generation Scripts feature install the Telematics Platform Software Development Kit in its setup, in the same way it should provide the set of images to be used as input for the update process.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Also, it is responsibility of the user to define its signature mechanism while the scripts provide facilities to create the minimal required files for integrity checks the scope and extension of the signature verification process and files is defined by the user.

It is expected that the user uses the folder structure stablished by the set_env.sh script so all the scripts can operate correctly to generate an ISO file with the required format by the Software Loading feature.

3.1.1.3 Extending Platform Functionality

User can modify the individual xml update package file generated by create_xml_descriptor.sh to include its own content type and images following the format defined by document TP_F101U-00028.

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Linux-based operating system configured for x86 or x86-64.

3.1.2.2 Setup Instructions

Follow the instructions to install the Telematics Platform Software Development Kit as described by TP_F101U-00035.

3.1.2.3 Setup Troubleshooting

See the Troubleshooting section of TP_F101U-00035.

3.1.3 Usage

3.1.3.1 Basic Instructions

Make sure that below environment variables are set:

- CAS_WORKDIR: Set to a folder where the Update Package file structure will be created. The Telematics Platform Software Development Kit User Guide provides instructions to set this variable
  - As an example, the variable can be set like this: export CAS_WORKDIR=$PWD
  - **!Note that above is just an example and user should determine which value to use.**
- CAS_HOSTFS: Set to a folder containing the Telematics Platform host tools. The Telematics Platform Software Development Kit User Guide provides instructions to set this variable
  - As an example, the variable can be set like this: export CAS_HOSTFS=${CAS_WORKDIR}/release-host/fs/host
  - **!Note that above is just an example and user should determine which value to use.**

!Note: In this section Figures are provided for illustrative purposes and it is not an exact representation of how these scripts should be used and what information is expected to be provided. For example: only two update files (boot.img and dsp2.ubifs) are used for generating an update package, this doesn't mean that those update files are mandatory nor it is limited to only them. Consult the Release Notes for a complete list of images to include in the update package.

Once the variables are set change directory to the location of the Software Loading Update Package Generation Scripts within the Software Development Kit.

- cd release/fs/devel/opt/conti/share/scripts/

Figure 1 Software Loading Update Package Generation Scripts shows the contents of the folder

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$
$ ls release/fs/devel/opt/conti/share/scripts/ -1
create_update_package.sh
create_xml_descriptor.sh
generate_diffs.sh
gen_update_descriptor.sh
set_env.sh
sign_files.sh
$
```

Figure 1 Software Loading Update Package Generation Scripts

3.1.3.2 Detailed Instructions

In order to generate an update package, the instructions provided in following sub-sections should be followed in sequential order.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.3.2.1 Script set_env.sh

The set environment script creates a source file that, when sourced, sets the environment variables required for the update package generation process. It also creates the working tree. Both the variables and directories are required for subsequent scripts. The working tree is defined as follows:

Directory Name	*Purpose*
base_files	The base files directory contains the initial files that will be used to generate differentials. These files should be the same as the ones already flashed in the target unit. **!Note: Users of this User Guide are responsible to provide the appropriate initial files.**
new_files	The new files directory contains the final files to apply to the base files for the differential. **!Note: Users of this User Guide are responsible to provide the appropriate final files.**
diff_update_files	The differential update files directory contains differentials created from the base and new directories.
non_diff_update_files	The non-differential update files directory contains updateable content to be updated by the platform and which don't require differentials. **!Note: Users of this User Guide are responsible to provide the appropriate update files.**
signature_files	This directory contains the signature files generated by the signature script or copied manually to this location.
additional_files	The additional files directory can contain any files necessary by an application to proceed with the update. **!Note: Users of this User Guide are responsible to provide the additional files.**
xml_files	The xml files directory contains the xml file descriptors for each platform updateable component as generated by the create_xml_descriptor.sh script.

© Continental Automotive GmbH 2019
Confidential and Proprietary

The following variables need to set before the set_env.sh script is run. **!Note that some of them have a default value while others won't and will fail if they are not set.**

Environment Variable	*Example Value*
UP_WORKSPACE_DIR (mandatory)	Your personal working directory, in this directory is where the folder structure is located. For example, *${CAS_WORKDIR}*/update_package_dir Default value is *${CAS_WORKDIR}*/update_package_dir
UP_RELEASE_HOST (mandatory)	Where the release-host tools are located. It is commonly located at *${CAS_WORKDIR}*/release-host It has no default value.
UP_ENV_FILE	Which contains the newly created variables. Default value is *${UP_WORKSPACE_DIR}*/up_environment
UP_NAME	The update package will be named after this variable. Default value is swl-test
UP_NEW_VERSION	Used only for informative purposes, it contains the target version. Default value is 2.0.1
UP_PREREQUISITES_VERSION	Used only for informative purposes, it contains the base version. Default value is 2.0.0
UP_DIFF_TOOL_PATH	Path to the xdelta3-pc opensource diff tool. Default value is *${UP_RELEASE_HOST}*/usr/bin
UP_UBICRC32_TOOL_PATH	CRC tool path. Default value is *${UP_RELEASE_HOST}*/usr/sbin
UP_OEM_VERSION	The overall version for the firmware update from the project team. Example: OTP-SA515M-3.1.5.27-DEVEL Default value is TP
UP_PRIVATE_KEY (optional)	This value is the private key used to digitally sign the image. Default value is *${UP_WORKSPACE_DIR}*/private_key.pem
UP_PUBLIC_KEY (optional)	This value is the public key used to retrieve the hash value from the digital signature. Default value is *${UP_WORKSPACE_DIR}*/public_key.pem
UP_HASH_TYPE (optional)	This value is hash technology used for the digital signature. Default value is SHA256. **!Note that. If signature process is used as delivered by the script, the value should remain as SHA256.**

© Continental Automotive GmbH 2019
Confidential and Proprietary

There are only two functions supported by the script.

1) Prepare environment and create folder structure: ./set_env.sh

Executed for the first time with no parameters, it will create the source file and the folder structure as shown in Figure 2 set_env.sh first execution

```
$
$ ./set_env.sh
 Start exporting environment
Variable is set: UP_WORKSPACE_DIR=/home/up_test/workspace/update_package_dir
Variable is set: UP_NAME=swl-test
Variable is set: UP_NEW_VERSION=2.0.1
Variable is set: UP_PREREQUISITES_VERSION=2.0.0
Variable is set: UP_DIFF_TOOL_PATH=/home/up_test/workspace/release/fs/host/usr/bin/
Variable is set: UP_UBICRC32_TOOL_PATH=/home/up_test/workspace/release/fs/host/usr/sbin/
Variable is set: UP_OEM_VERSION=TP
Variable is set: UP_UPGRADE_MODE=ssu
Variable is set: UP_PRIVATE_KEY=/home/up_test/workspace/update_package_dir/private_key.pem
Variable is set: UP_PUBLIC_KEY=/home/up_test/workspace/update_package_dir/public_key.pem
Variable is set: UP_HASH_TYPE=SHA256
Creating dir /home/up_test/workspace/update_package_dir/base_files/
Creating dir /home/up_test/workspace/update_package_dir/new_files/
Creating dir /home/up_test/workspace/update_package_dir/diff_update_files/
Creating dir /home/up_test/workspace/update_package_dir/non_diff_update_files/
Creating dir /home/up_test/workspace/update_package_dir/signature_files/
Creating dir /home/up_test/workspace/update_package_dir/additional_files/
Creating dir /home/up_test/workspace/update_package_dir/xml_files/
Execute below command to set the right environment
source /home/up_test/workspace/up_environment
$
```

Figure 2 set_env.sh first execution

After that, if the script is executed again and the folder structure hasn't been deleted, it will create the source file but ignore the file structure creation as shown in Figure 3 set_env.sh subsequent execution

```
$ ./set_env.sh
 Start exporting environment
Variable is set: UP_WORKSPACE_DIR=/home/up_test/workspace/update_package_dir
Variable is set: UP_NAME=swl-test
Variable is set: UP_NEW_VERSION=2.0.1
Variable is set: UP_PREREQUISITES_VERSION=2.0.0
Variable is set: UP_DIFF_TOOL_PATH=/home/up_test/workspace/release/fs/host/usr/bin/
Variable is set: UP_UBICRC32_TOOL_PATH=/home/up_test/workspace/release/fs/host/usr/sbin/
Variable is set: UP_OEM_VERSION=TP
Variable is set: UP_UPGRADE_MODE=ssu
Variable is set: UP_PRIVATE_KEY=/home/up_test/workspace/update_package_dir/private_key.pem
Variable is set: UP_PUBLIC_KEY=/home/up_test/workspace/update_package_dir/public_key.pem
Variable is set: UP_HASH_TYPE=SHA256
Skiping [ /home/up_test/workspace/update_package_dir/base_files/ ] directory already exist
Skiping [ /home/up_test/workspace/update_package_dir/new_files/ ] directory already exist
Skiping [ /home/up_test/workspace/update_package_dir/diff_update_files/ ] directory already exist
Skiping [ /home/up_test/workspace/update_package_dir/non_diff_update_files/ ] directory already exist
Skiping [ /home/up_test/workspace/update_package_dir/signature_files/ ] directory already exist
Skiping [ /home/up_test/workspace/update_package_dir/additional_files/ ] directory already exist
Skiping [ /home/up_test/workspace/update_package_dir/xml_files/ ] directory already exist
Execute below command to set the right environment
source /home/up_test/workspace/up_environment
$
```

Figure 3 set_env.sh subsequent execution

© Continental Automotive GmbH 2019
Confidential and Proprietary

!Note that once the execution has finished it is required to execute the source command as indicated in the output.

- source /home/up_test/workspace/up_environment

After above commands have been executed the environment should be ready for the next scripts execution.

!Note that source command has only effect in the currently open terminal window. If a new terminal window is open the source command should be executed again.

2) Clean folder structure: ./set_env.sh --clean

When executed it will clear the contents of the update package file structure. It is a simple command that doesn't return any error code nor error messages, but users can verify that the operation was done successfully by checking the contents of the update package file structure as shown in Figure 4 set_env.sh clean operation

```
$./set_env.sh --clean
$find /home/up_test/workspace/update_package_dir/
/home/up_test/workspace/update_package_dir/
/home/up_test/workspace/update_package_dir/new_files
/home/up_test/workspace/update_package_dir/additional_files
/home/up_test/workspace/update_package_dir/base_files
/home/up_test/workspace/update_package_dir/diff_update_files
/home/up_test/workspace/update_package_dir/signature_files
/home/up_test/workspace/update_package_dir/non_diff_update_files
/home/up_test/workspace/update_package_dir/xml_files
$
```

Figure 4 set_env.sh clean operation

3.1.3.2.2 Script generate_diffs.sh

!Note that the execution of this script is optional. It is only required when there are two set of images (base and target) and a diff update image generation is required.

This script uses the *${BASE_FILES_DIR}*, which contains the existing images, and the *${NEW_FILES_DIR}*, which contains the final images, to create differentials which are stored at the *${DIFF_UPDATE_FILES_DIR}*. Currently this script only supports xdelta opensource technology.

This script only supports one operation

1) Differential files generation: ./generate_diffs.sh --gen_diffs

In order to execute successfully above command, the user should provide two set of images.

1) Base images: set of images where the differential update is expected to be applied. Its folder is *${UP_WORKSPACE_DIR}*/base_files
2) New images: set of images will be the result of applying the differential update on top of the base images. Its folder is *${UP_WORKSPACE_DIR}*/new_files

Both set of images should be provided by user and placed in the appropriate folder within the update package files structure as shown in Figure 5 generate_diffs.sh successful diff generation in order to generate files with extension .diff for each pair of images where the name matches.

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$./generate_diffs.sh --gen_diffs
generate_diffs.sh Generating diff files ...
get_files ...
######## Looking for differences on update elements or full images...
2 files present under /home/up_test/workspace/update_package_dir/new_files/: boot.img dsp2.ubifs
2 files present under /home/up_test/workspace/update_package_dir/base_files/: boot.img dsp2.ubifs
removing /home/up_test/workspace/update_package_dir/diff_update_files/..
generate_diff_file ...
Generation diff image using xDelta for:boot
Generation diff image using xDelta for:dsp2
$ls /home/up_test/workspace/update_package_dir/diff_update_files/
boot.diff  dsp2.diff
$
```

Figure 5 generate_diffs.sh successful diff generation

!Note that when both set of images are the same, the generate_diff.sh script will not generate any diff file as shown in Figure 6 generate_diffs.sh stopped diff generation

```
$cp release/images/devel/4K/dsp2.ubifs update_package_dir/base_files/
$cp release/images/devel/4K/boot.img update_package_dir/base_files/
$cp release/images/devel/4K/boot.img update_package_dir/new_files/
$cp release/images/devel/4K/dsp2.ubifs update_package_dir/new_files/
$cd release/fs/devel/opt/conti/share/scripts/
$./generate_diffs.sh --gen_diffs
generate_diffs.sh Generating diff files ...
get_files ...
######## Looking for differences on update elements or full images...
2 files present under /home/up_test/workspace/update_package_dir/new_files/: boot.img dsp2.ubifs
2 files present under /home/up_test/workspace/update_package_dir/base_files/: boot.img dsp2.ubifs
generate_diffs.sh No candidates to generate diff files, stopping execution ...
$
```

Figure 6 generate_diffs.sh stopped diff generation

As a final note, we should consider that the execution of this script is only required when diff updates are supported. In case the project requires to use full updates (applying the whole image directly to the unit). The files should be placed in the *${UP_WORKSPACE_DIR}*/non_diff_update_files folder as shown in Figure 7 non_diff_updates_files

```
$cp release/images/devel/4K/boot.img update_package_dir/non_diff_update_files/
$cp release/images/devel/4K/dsp2.ubifs update_package_dir/non_diff_update_files/
$
```

Figure 7 non_diff_updates_files

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.3.2.3 Script create_xml_descriptor.sh

This script creates individual XML descriptors for each updatable element located at diff_update_files and non_diff_update_files folders. If it is needed the xml definitions can be modified to add extra information.

As reference here is the tag description that can be modified.

Tag	*Description*
size	Size of the file
newromsize	Size of the new version file (in case of full update file it will be the same as size).
signature name	Name of the signature (It shouldn't include the path).
type	Type of updatable element (full or diff).
contenttype	defined by the Update Agent implementation
updatemode	The installation mode of the element (normal or recovery).
dependentfiles	The files that will be needed by this update element.
version	New version included in the update element.
checksum	Mathematical value computed (It shouldn't be modified).
prereqversion	The version that is needed to update this element.

This script only supports one operation

1) Descriptor files generation: ./create_xml_descriptor.sh --gen_descriptors

Which basically generates a set of individual xml files containing the metadata needed for an update package as shown in Figure 8 create_xml_descriptor.sh individual xml generation

```
$./create_xml_descriptor.sh --gen_descriptors
create_xml_descriptor.sh Generating XML descriptor files ...
check_for_mtd_files ...
---Processing boot.diff file---
get_image_name_and_type ...
get_image_type ...
get_size_and_new_rom_size ...
get_content_type ...
calculate_checksum ...
create_xml_descriptor ...
----------------------------------
---Processing dsp2.diff file---
get_image_name_and_type ...
get_image_type ...
get_size_and_new_rom_size ...
get_content_type ...
calculate_checksum ...
create_xml_descriptor ...
----------------------------------
```

Figure 8 create_xml_descriptor.sh individual xml generation

© Continental Automotive GmbH 2019
Confidential and Proprietary

These files are stored in *${UP_WORKSPACE_DIR}*/xml_files/ so they can be modified according to the user product needs.

!Note: Persistence update file name for external application processor must end with _extap (e.g. persistent_update_extap.xml); and persistence update file name for modem application processor must end with _modemap (e.g. persistent_update_modemap.xml).

3.1.3.2.4 Script gen_update_descriptor.sh

This script concatenates all the individual xml descriptors in one file and will erase all the individual ones.

This script only supports one operation

1) Merging individual xml files: ./gen_update_descriptor.sh --join_descriptors

Which basically generates the final update package xml file as shown in Figure 9 gen_update_descript.sh join descriptors that will serve as input for the Software Loading Manager and Update Agents to perform the update process with the images contained in the ISO file.

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$./gen_update_descriptor.sh --join_descriptors
gen_update_descriptor.sh Joining XML descriptors ...
<?xml version="1.0" encoding="UTF-8"?>
<!-- Copyright (c) 2016 Continental AG and subsidiaries                    -->
<!--                                                                       -->
<!-- The copyright notice above does not evidence any actual               -->
<!-- or intended publication of such source code.                          -->
<!-- The code contains Continental Confidential Proprietary Information. -->
<packagecontent numberimages="2">
        <oem version="TP"/>
        <image name="boot" filename="boot.diff" type="mtd">
                <size>3895828</size>
                <newromsize>5183488</newromsize>
                <contenttype>boot</contenttype>
                <type>diff</type>
                <dependentfiles>
                <!-- Set the dependent files here -->
                        <file>boot.xml</file>
                </dependentfiles>
                <version>2.0.1</version>
                <checksum>0x35d85d91</checksum>
                <prerequisiteversion>2.0.0</prerequisiteversion>
                <!-- Set this to tag to normal if needed -->
                <updatemode>normal</updatemode>
                <!-- Add project specific tags here -->
                </image>
        <image name="dsp2" filename="dsp2.diff" type="ubifs">
                <size>277</size>
                <newromsize>62726144</newromsize>
                <contenttype>dsp2</contenttype>
                <type>diff</type>
                <dependentfiles>
                <!-- Set the dependent files here -->
                        <file>dsp2.xml</file>
                </dependentfiles>
                <version>2.0.1</version>
                <checksum>0xd592c677</checksum>
                <prerequisiteversion>2.0.0</prerequisiteversion>
                <!-- Set this to tag to normal if needed -->
                <updatemode>normal</updatemode>
                <!-- Add project specific tags here -->
                </image>
</packagecontent>
Generated update package descriptor content:
```

Figure 9 gen_update_descript.sh join descriptors

3.1.3.2.5 Script sign_files.sh

The usage of this script is optional since the user can determine the most convenient tool for its project needs to generate the signatures, but this script can help for testing purposes.

There are only two functions supported by the script.

1) Key pairs creation: ./sign_files.sh --create_keys

If you execute the script with this parameter, the script will create public and private RSA keys in the signature_files folder as shown in Figure 10 sign_files.sh key creation

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$./sign_files.sh --create_keys
sign_files.sh Creating RSA keys ...
create_rsa_keys ...
Generating RSA private key, 1024 bit long modulus (2 primes)
...+++++
....+++++
e is 65537 (0x010001)
writing RSA key
```

Figure 10 sign_files.sh key creation

The keys are created on *${UP_WORKSPACE_DIR}* folder.

2) File signing: ./sign_files.sh --sign_files

If you execute the script with this parameter, the script will sign the files located at the diff_update_files, non_diff_update_files, additional_files folders using the RSA keys previously generated. See below:

- *${UP_PRIVATE_KEY}* is used to sign the contents of diff_update_files, non_diff_update_files and additional_files folders.
- *${UP_PUBLIC_KEY}* is used to verify the image on target.
    - For security reasons none of the keys are included in the update package.
    - The user is responsible of making sure that the public key is available in target before the update process is started.

It will also sign the update_package_descriptor.xml as shown in Figure 11 sign_files.sh file signing

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$./sign_files.sh --sign_files
sign_file: boot.diff ...
sign_file: dsp2.diff ...
/home/up_test/workspace/release/fs/devel/opt/conti/share/scripts
<?xml version="1.0" encoding="UTF-8"?>
<!-- Copyright (c) 2016 Continental AG and subsidiaries                    -->
<!--                                                                       -->
<!-- The copyright notice above does not evidence any actual               -->
<!-- or intended publication of such source code.                          -->
<!-- The code contains Continental Confidential Proprietary Information. -->
<packagecontent numberimages="2">
        <oem version="TP"/>
        <image name="boot" filename="boot.diff" type="mtd">
                <size>3895828</size>
                <newromsize>5183488</newromsize>
                <contenttype>boot</contenttype>
                <type>diff</type>
                <dependentfiles>
                <!-- Set the dependent files here -->
                        <file>boot.xml</file>
                </dependentfiles>
                <version>2.0.1</version>
                <checksum>0x35d85d91</checksum>
                <prerequisiteversion>2.0.0</prerequisiteversion>
                <!-- Set this to tag to normal if needed -->
                <updatemode>normal</updatemode>
                <!-- Add project specific tags here -->
                </image>
        <image name="dsp2" filename="dsp2.diff" type="ubifs">
                <size>277</size>
                <newromsize>62726144</newromsize>
                <contenttype>dsp2</contenttype>
                <type>diff</type>
                <dependentfiles>
                <!-- Set the dependent files here -->
                        <file>dsp2.xml</file>
                </dependentfiles>
                <version>2.0.1</version>
                <checksum>0xd592c677</checksum>
                <prerequisiteversion>2.0.0</prerequisiteversion>
                <!-- Set this to tag to normal if needed -->
                <updatemode>normal</updatemode>
                <!-- Add project specific tags here -->
                </image>
        <signatures>
                <file name="boot.diff" signature="boot.diff.sig"/>
                <file name="dsp2.diff" signature="dsp2.diff.sig"/>
        </signatures>
</packagecontent>
sign_file: update_package_descriptor.xml ...
```

Figure 11 sign_files.sh file signing

3.1.3.2.6 Script create_update_package.sh

There are only two functions supported by the script.

1) Create ISO: ./create_update_package.sh --create_iso_up

This script generates an ISO file containing the images to update, xml metadata files and signature files for the update process as shown in Figure 12 create iso update package

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$./create_update_package.sh --create_iso_up
create_update_package.sh Creating ISO update package file ...
Warning: creating filesystem that does not conform to ISO-9660.
Warning: ISO-9660 filenames longer than 31 may cause buffer overflows in the OS.
Warning: creating filesystem with Joliet extensions but without Rock Ridge
         extensions. It is highly recommended to add Rock Ridge.
I: -input-charset not specified, using utf-8 (detected in locale settings)
Total translation table size: 0
Total rockridge attributes bytes: 0
Total directory bytes: 0
Path table size(bytes): 10
Max brk space used 0
2088 extents written (4 MB)
$ls /home/up_test/workspace/update_package_dir/
additional_files  base_files  diff_update_files  new_files  non_diff_update_files  private_key.pem  public_key.pem  signature_files  swl-test.iso  xml_files
```

Figure 12 create iso update package

2) Create ISO without joliet support: ./create_update_package.sh --create_iso_up --no_joliet

If you execute the script with this parameter, the script creates an ISO file containing the images to update, xml metadata files and signature files but without upper case support as shown in Figure 13 create iso update package with no joliet

```
$./create_update_package.sh --create_iso_up --no_joliet
create_update_package.sh Creating ISO update package file ...
Warning: creating filesystem that does not conform to ISO-9660.
Warning: ISO-9660 filenames longer than 31 may cause buffer overflows in the OS.
I: -input-charset not specified, using utf-8 (detected in locale settings)
 14.52% done, estimate finish Thu Nov  7 01:25:53 2019
 29.06% done, estimate finish Thu Nov  7 01:25:53 2019
 43.54% done, estimate finish Thu Nov  7 01:25:55 2019
 58.07% done, estimate finish Thu Nov  7 01:25:54 2019
 72.56% done, estimate finish Thu Nov  7 01:25:54 2019
 87.09% done, estimate finish Thu Nov  7 01:25:54 2019
Total translation table size: 0
Total rockridge attributes bytes: 0
Total directory bytes: 0
Path table size(bytes): 10
Max brk space used 0
34462 extents written (67 MB)
$ls /home/up_test/workspace/update_package_dir/
additional_files  diff_update_files  non_diff_update_files  public_key.pem   swl-test_no-joliet.iso
base_files        new_files          private_key.pem        signature_files  xml_files
$
```

Figure 13 create iso update package with no joliet

At this point the update package is created in *${UP_WORKSPACE_DIR}* folder with ISO extension and can be used for the update process.

3.1.3.2.7 Putting it all together

As reference attached is a text file with the console output after executing this process with a pair of release images.

- foo_v1 is the base image
- foo_v2 is the target image

Console Output:
update_package_generation_execution.txt

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.3.3 Preparing package for persistence update agent

3.1.3.3.1 Using “*.tar.gz” archives for persistence update

- For ExtAp, copy "tp-pers-install.tar.gz" from ExtAp image folder to "non_diff_update_files" folder, rename file "tp-pers_extap.tar.gz"
- For ModemAp, copy "tp-pers-install.tar.gz" from ModemAp image folder to "non_diff_update_files" folder, rename file "tp-pers_modemap.tar.gz"

```
$ll update_package_dir/non_diff_update_files/
итого 24
drwxr-xr-x 2 vadim vadim 4096 Мар 25 16:58 ./
drwxr-xr-x 9 vadim vadim 4096 Мар 25 17:03 ../
-rw-r--r-- 1 vadim vadim 4217 Мар 25 16:58 tp-pers_extap.tar.gz
-rw-r--r-- 1 vadim vadim 4217 Мар 25 16:58 tp-pers_modemap.tar.gz
$
```

Figure 14 placing both persistence archives together

- Create ISO: ./create_update_package.sh --create_iso_up

```
$grep -B 4 persistent-config update_package_dir/xml_files/update_package_descriptor.xml
        <oem version="TP" />
        <image name="tp-pers_extap.tar.gz" filename="tp-pers_extap.tar.gz" type="">
                <size>4217</size>
                <newromsize>4217</newromsize>
                <contenttype>persistent-config-extap</contenttype>
--
                </image>
        <image name="tp-pers_modemap.tar.gz" filename="tp-pers_modemap.tar.gz" type="">
                <size>4217</size>
                <newromsize>4217</newromsize>
                <contenttype>persistent-config-nad</contenttype>
$
```

Figure 15 persistence archive entries at xml descriptor file

3.1.3.3.2 Using XML files for persistence update

3.1.3.3.2.1 Prepare ExtAp files

1. Unarchive "tp-pers-install.tar.gz" from EXTAP image folder to temporary folder
2. Copy "GENERATED_pers_combined_config.xml" to "non_diff_update_files" folder, rename file to "GENERATED_pers_combined_config_extap.xml"

3.1.3.3.2.2 Prepare ModemAp files

1. Unarchive "tp-pers-install.tar.gz" from MODEMAP image folder to temporary folder
2. Copy "GENERATED_pers_combined_config.xml" to "non_diff_update_files" folder, rename file to "GENERATED_pers_combined_config_modemap.xml"

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$ll update_package_dir/non_diff_update_files/
итого 56
drwxr-xr-x 2 vadim vadim  4096 Мар 25 17:16 ./
drwxr-xr-x 9 vadim vadim  4096 Мар 25 17:17 ../
-rw-r--r-- 1 vadim vadim 21382 Мар 25 17:16 GENERATED_pers_combined_config_extap.xml
-rw-r--r-- 1 vadim vadim 21382 Мар 25 17:16 GENERATED_pers_combined_config_modemap.xml
$
```

Figure 16 placing both persistence xml entries together

- Create ISO: ./create_update_package.sh --create_iso_up

```
$grep -B 4 persistent-config update_package_dir/xml_files/update_package_descriptor.xml
        <oem version="TP" />
        <image name="GENERATED_pers_combined_config_extap.xml" filename="GENERATED_pers_combined_config_extap.xml" type="">
                <size>21382</size>
                <newromsize>21382</newromsize>
                <contenttype>persistent-config-extap</contenttype>
--
                </image>
        <image name="GENERATED_pers_combined_config_modemap.xml" filename="GENERATED_pers_combined_config_modemap.xml" type="">
                <size>21382</size>
                <newromsize>21382</newromsize>
                <contenttype>persistent-config-nad</contenttype>
$
```

Figure 17 persistence xml entries at xml descriptor file

3.1.3.4 Testing/Error Reporting

Not Applicable

3.2 Software Download Blocks Generation

3.2.1 Overview

3.2.1.1 Platform Content Overview

The Software Loading feature also provides an upper layer to generate multiple Software Download Blocks (Update Packages) at once based on a configuration file. This layer consists in a script that internally uses the Update Package Generation Scripts.

This Software Download Blocks Generation script can be used to generate the Update Package(s) needed for an update campaign. This script is included as part of the Telematics Platform Software Development Kit.

3.2.1.2 Product Expectations

It is expected that users of Telematics Platform Software Loading Software Download Blocks Generation Script feature install the Telematics Platform Software Development Kit in its setup. Users can modify the input configuration file (Section 3.2.3.2.2) to include its own update elements to be used during the update process. This may require modifying the XML descriptor file to have the proper attributes.

!Note: This is only expected to work with full images not delta.

3.2.1.3 Extending Platform Functionality

Not Applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

- Python (Tested with 2.7.12 and 3.5.2 versions)

© Continental Automotive GmbH 2019
Confidential and Proprietary

- Linux-based operating system configured for x86 or x86-64

3.2.2.2 Setup Instructions

Follow the instructions to install the Telematics Platform Software Development Kit as described by TP_F101U-00035.

3.2.2.3 Setup Troubleshooting

See the Troubleshooting section of TP_F101U-00035.

3.2.3 Usage

3.2.3.1 Basic Instructions

Since this script internally uses the Update Package Generation Scripts it is necessary to follow their usage instructions as described in Section 3.1.3.1.

3.2.3.2 Detailed instructions

3.2.3.2.1 Parameters

The script accepts five parameters, two of them are mandatory and the other ones are optional. If an optional parameter is not provided, the default value will be used.

These are the script parameters:

Parameter	*Description*	*Optional*	*Default Value*
Images Directory (-i)	Folder that contains the images	No	N/A
Output Directory (-o)	Folder in which the SWDL block will be placed.	No	N/A
Blocks Config File (-b)	JSON Configuration file that contains the information about the blocks content.	Yes	<CAS_WORK_DIR>/.build/config/swdl_blocks.json
SWL Scripts Directory (-s)	Folder in which the Update Package Generation Scripts are located.	Yes	<CAS_WORK_DIR>/package/conti/conti-swl/scripts/
Environment Configuration File (-e)	Config file generated by the set_env.sh execution. See section 3.1.3.2.1.	Yes	<CAS_WORK_DIR>/up_environment

3.2.3.2.2 Blocks config file

The script requires an input configuration file in JSON format. This format consists in “block_name” and “content” attributes for each of the blocks.

Attribute	*Description*
block_name	This will be used to name the SWDL block
content	A set of filenames that will be included in the SWDL block. These files must be in the images directory that is given as a parameter.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Here is an example:

```
[
  {
    "block_name": "Block_1",
    "content": [
    "abl.elf",
    "aop.mbn",
    "apdp.mbn",
    "boot.img",
    "hyp.mbn",
    "multi_image.mbn",
    "sbl1.mbn",
    "tz.mbn",
    "uefi.elf"
    ]
  },
  {
    "block_name": "Block_2",
    "content": [
    "system.ubifs",
    "dsp2.ubifsi"
    ]
  },
  {
    "block_name": "Block_3",
    "content": [
    "tp_modem_config_list.tar.gz",
    "tp-pers-install.tar.gz"
    ]
  }
]
```

Figure 154 SWDL blocks JSON file

!Note: All the filenames listed in Figure 14 are just for reference.

As reference, attached is a swdl blocks config file that can be used as a template:

swdl_blocks.json

!Notes:

i. If the update campaign is intended for multiple processors, there must be one JSON config file for each of the processors.
ii. The file attached is only a template and must be modified according to the project needs.

3.2.3.2.3 Executing the script

This script allows the user to execute it in two ways:

- From the console itself
- As an imported python module

Here is an example of how to execute it from the console with no optional parameters:

```
python generate_swdl_blocks.py -o /Shared/output/ -i /Shared/4K/
```

And with all the parameters:

```
python generate_swdl_blocks.py -o /Shared/output/ -i /Shared/4K/ -b swdl_blocks.json -
s ~/workspace/thick/package/conti/conti-swl/scripts -e
~/workspace/thick/up_environment
```

For the execution as an imported module, the module must me imported:

```
from generate_swdl_blocks import create_swdl_blocks
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

This is the API definition:

```
create_swdl_blocks(output_dir, image_dir, blocks_file, swl_scripts, env)
```

!Note: The parameters must be provided in this specific order

Here is an example of how to invoke it with no optional parameters:

```
#!/usr/bin/env python3

from generate_swdl_blocks import create_swdl_blocks


ret = create_swdl_blocks("/Shared/output/", "/Shared/4K/")
if (ret !=0):
    print("SWDL blocks generation failed")
else:
    print("SWDL Blocks were generated successfully")
```

And this is how to invoke it with optional parameters:

```
#!/usr/bin/env python3

from generate_swdl_blocks import create_swdl_blocks


ret = create_swdl_blocks("/Shared/output/", "/Shared/4K/", "swdl_blocks.json",
"~/workspace/thick/package/conti/conti-swl/scripts", "~/workspace/thick/up_environment")
if (ret !=0):
    print("SWDL blocks generation failed")
else:
    print("SWDL Blocks were generated successfully")
```

Once the script execution finishes, the output directory will contain one update package (iso) per block.

Name	Size	Type	Modified
Block_1.iso	550.9 kB	Unknown	19:38
Block_2.iso	11.4 MB	Unknown	19:37
Block_3.iso	128.9 MB	Unknown	19:38

Figure 165 Expected output

These iso files are intended to be used to perform an update using SWL capability. For more information please refer to document TP_F101U-00028.

3.2.3.3 Testing/Error Reporting

Not Applicable

4 Known Issues

Not Applicable

5 FAQs

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

6 Troubleshooting/Tips and Tricks

- Issue: bash: ./set_env.sh: /bin/bash: bad interpreter: No such file or directory
    a. Solution: based on the linux distribution, used the packaging system that contains bash binary
    b. **!Note: above message used as example set_env.sh but it can happen with any of the Software Loading Update Package Generation Scripts.**

7 References

7.1 Abbreviations/Acronyms/Definitions

Not Applicable

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary

TP_F101U-00028: Software Loading User Guide

TP_F101U-00035: Software Development Kit User Guide

TP_F100W-00015: Update Agent Writers Guide

7.3 Website links

http://xdelta.org/
https://www.json.org/

© Continental Automotive GmbH 2019
Confidential and Proprietary

8 Revision History

Version	Date	Description
01.00	Nov 25, 2019	Initial Release
02.00	Dec 24, 2020	Add persistence update xml naming convention
03.00	Sep 11, 2020	Add more details about user expectations
04.00	Nov 20, 2020	Add SWDL blocks generation information
05.00	Apr 8, 2021	Add generation steps of persistence update package with tar.gz or xml
		== ConMod-specific content from here ==

© Continental Automotive GmbH 2019
Confidential and Proprietary